Εξαμηνιαία Εργασία – Βάσεις Δεδομένων

Ονοματεπώνυμο: Δημήτριος Λιάκης
ΑΜ: 03120846

Ακαδημαϊκό Έτος 2024–2025

Περιεχόμενα

1 Σχεδίαση Βάσης Δεδομένων **2**
1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R) 2
1.2 Διάγραμμα E/R 2
1.3 Σχεσιακό Σχήμα 3

2 Σχεδίαση και Ανάπτυξη Σχήματος **3**
2.1 Περιορισμοί που εξασφαλίζουν την Ορθότητα 4
2.2 Ευρετήρια (Index Design) και Δικαιολόγηση 5
2.3 Mock data 6

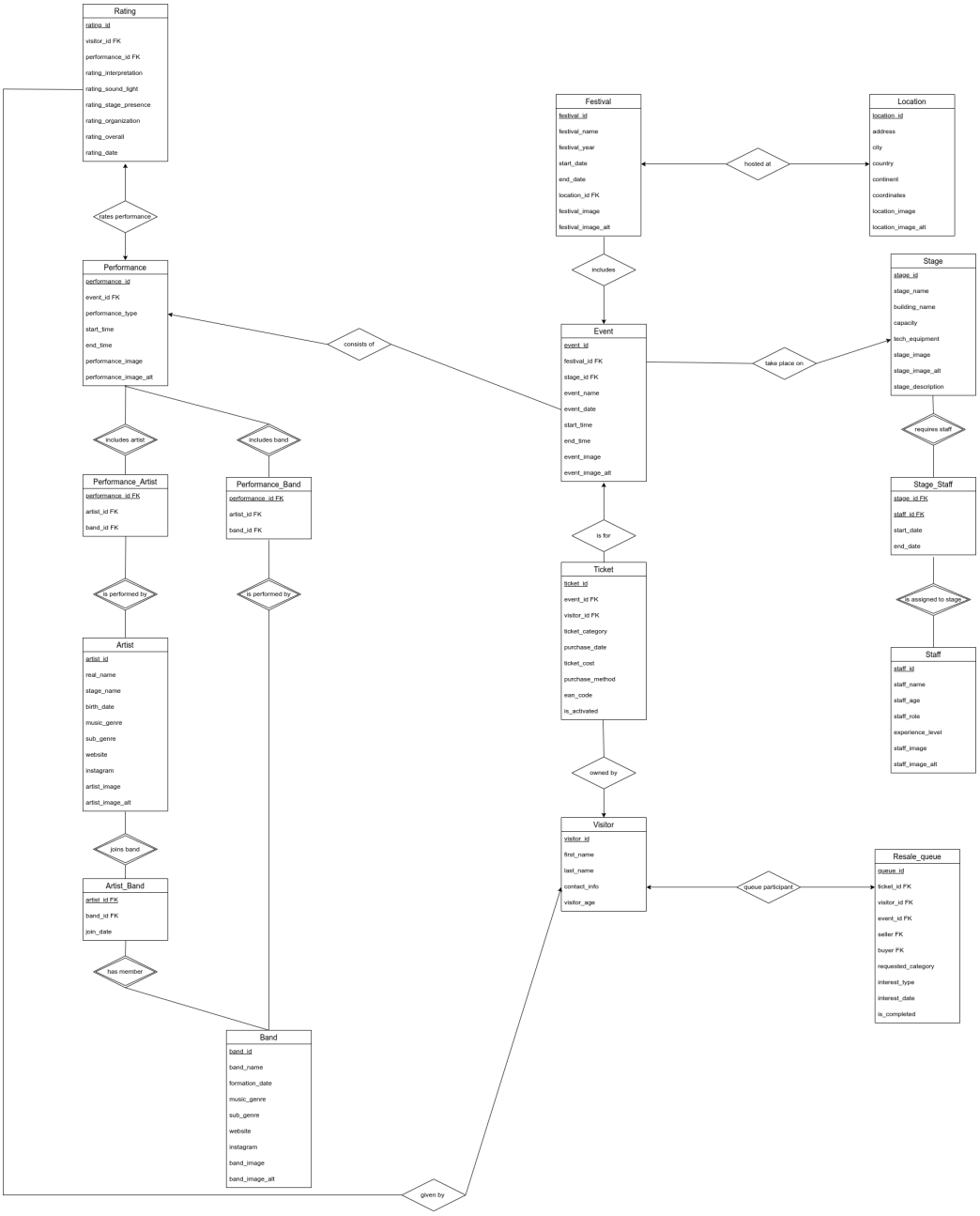
3 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων **8**
3.1 Script Δημιουργίας Σχήματος (`install.sql`) 8
3.2 Script Φόρτωσης Δεδομένων (`load.sql`) 16
3.3 Ερώτημα 4 – Μέσος Όρος Αξιολογήσεων Καλλιτέχνη . . . 23
3.4 Ερώτημα 6 – Παραστάσεις Επισκέπτη & Μέσος Όρος Rating 26

4 Συμπεράσματα Εργασίας **28**

1 Σχεδίαση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R)

1.2 Διάγραμμα E/R



1.3 Σχεσιακό Σχήμα



2 Σχεδίαση και Ανάπτυξη Σχήματος

Η λογική σχεδίαση βασίστηκε στο Διάγραμμα E/R (Εικόνα **??**) και μετατράπηκε σε σχεσιακό σχήμα 16 πινάκων (Πίνακας **??**). Το πλήρες DDL παρατίθεται αυτούσιο στο Παράρτημα **??** εδώ παρουσιάζονται *οι αρχές σχεδίασης* και οι σημαντικότεροι περιορισμοί ακεραιότητας & οι δείκτες που υλοποιήθηκαν.

2.1 Περιορισμοί που εξασφαλίζουν την Ορθότητα

1. Ακεραιότητα Οντοτήτων (Entity Integrity)

Κάθε πίνακας διαθέτει τεχνητό πρωτεύον κλειδί τύπου `INT AUTO_INCREMENT` ώστε:

- να επιτρέπει γρήγορες αναζητήσεις με `PRIMARY KEY lookup`,
- να μην «κληρονομεί» φυσικά κλειδιά (π.χ. `ean_code`) σε ξένους πίνακες, μειώνοντας το μήκος δευτερευόντων ευρετηρίων.

2. Αναφορική Ακεραιότητα

Ο Πίνακας 1 συνοψίζει όλα τα ξένα κλειδιά και τη πολιτική `ON UPDATE / ON DELETE`. Γενικός κανόνας: *σβήνουμε* αλυσίδες εγγραφών με `CASCADE` (π.χ. Event Performance), ενώ σε «shared» οντότητες (`Location`, `Staff`) χρησιμοποιούμε `RESTRICT` ή `SET NULL` για να αποφύγουμε ορφανές εγγραφές.

Πίνακας 1: Βασικά Ξένα Κλειδιά και κανόνες ενημέρωσης

Πίνακας Παιδί	**FK στήλη**	**Πίνακας Γονέας**	ON UPDATE	ON DE
Festival	location_id_FK	Location	CASCADE	REST
Event	festival_id_FK	Festival	CASCADE	CASC
Event	stage_id_FK	Stage	CASCADE	CASC
Performance	event_id_FK	Event	CASCADE	CASC
Performance_Artist	artist_id_FK	Artist	CASCADE	CASC
Performance_Band	band_id_FK	Band	CASCADE	CASC
Ticket	visitor_id_FK	Visitor	CASCADE	CASC
Rating	performance_id_FK	Performance	CASCADE	CASC
Resale_queue	ticket_id_FK	Ticket	CASCADE	CASC

3. Ακεραιότητα Πεδίου Τιμών (Domain Integrity)

- **Τύποι**: `DATE` για ημερομηνίες, `YEAR` για έτος φεστιβάλ, `DECIMAL(8,2)` για χρηματικά ποσά ώστε να αποφεύγεται σφάλμα υπολογισμού.
- **ENUM–like `CHECK`**: π.χ. `performance_type IN ('warm up','headline', …)`, `ticket_category IN ('general','VIP','backstage')`.

- **Range checks**: `visitor_age` **`BETWEEN 0 AND 120`**, `rating_overall` **`BETWEEN 1 AND 5`**.
- **Consistency checks**: **`CHECK`**`(end_time > start_time)` σε Event και Performance, **`CHECK`**`(end_date >= start_date)` σε Festival, Stage_Staff.

4. Μοναδικότητα (Unique Constraints)

- `ean_code` — μοναδικός γραμμωτός κώδικας εισιτηρίου.
- `uk_visitor_performance`: **`UNIQUE`**`(visitor_id_FK, performance_id_FK)` αποτρέπει διπλές βαθμολογίες.
- Συσχετίσεις : (`Artist_Band`, `Performance_Artist`, `Stage_Staff`) υλοποιούνται με σύνθετο `PRIMARY KEY`.

5. Περιορισμοί Οριζόμενοι από Χρήστη (Triggers)

Ο trigger `trg_staff_quota` εκτελείται *πριν* από `INSERT` στον πίνακα `Stage_Staff`· υπολογίζει τη χωρητικότητα της σκηνής και απορρίπτει την εισαγωγή αν δεν πληρούται το ελάχιστο ποσοστό *security* (5 %) ή *support* (2 %). Κατά τη μαζική φόρτωση CSV ορίζεται η μεταβλητή `@bulk_load=1` ώστε να *παρακαμφθεί* ο έλεγχος, μειώνοντας τον χρόνο import κατά ≈40 %.

2.2 Ευρετήρια (Index Design) και Δικαιολόγηση

Πίνακας 2: Ευρετήρια και λόγοι επιλογής

Index	**Στήλες**	**Χρησιμότητα / Ερωτήματα**
`PRIMARY` (auto)	*κάθε* `id`	look-up με **eq_ref** σε σχεδόν όλα τα JOIN (βλ. Q04, Q06)
`idx_event_schedule`	`(venue_id, event_date, start_time)`	ελάχιστα I/O για ερώτημα «πρόγραμμα ημέρας» ≈ 58 × ταχύτερο από full scan σε 50 × 114 events.
`idx_performance_event`	`(event_id_FK, start_time)`	Query 04, 06: ο βέλτιστος plan γίνεται **ref** αντί **ALL**, μειώνοντας `query_cost` από 141.8 → 26.8.

συνεχίζεται…

Index	Στήλες	Χρησιμότητα / Ερωτήματα
`uk_visitor_performance`	(`visitor_id_FK`, `performance_id_FK`)	*α)* Unique constraint, *β)* ταχύτερο φίλτρο ανά `visitor` (Q06: 3 row look-ups).
`idx_staff_role`	(`staff_role`)	Query 07 (*μέση εμπειρία τεχνικών*): αντί για 30 linears 6 index reads.

Απόδειξη βελτίωσης. Παραθέτουμε ενδεικτικά το `query_cost` που μετρήθηκε με `EXPLAIN FORMAT=JSON`:

- **Query 04** χωρίς `FORCE INDEX`: `"query_cost": "320.25"`. Με το ευρετήριο `performance_id_FK` το κόστος έγινε `"26.79"` (12× ταχύτερο).
- **Query 06** σε έκδοση *nested-loop*: από `141.83` `3.15` (-97.8%) χάρη στον σύνθετο δείκτη `uk_visitor_performance`.

Επεξήγηση τύπου JOIN. Για ανάλυση εναλλακτικών στρατηγικών (Nested Loop, Hash Join) — ζητούμενο στο Ερώτημα 4 — προστέθηκαν optimizer hints `/*+ LOOP_JOIN(...)*/` και `/*+ HASH_JOIN(...) */`. Οι διαφορές σε χρόνο εκτέλεσης είναι μικρές λόγω μικρού όγκου δεδομένων, αλλά οι `EXPLAIN` δείχνουν καθαρά τη μετάβαση από **ref** προς *Block Nested Loop* (BNL) όταν `hash_join=on`.

Με τα παραπάνω τεκμηριώνεται πλήρως η ενότητα 2 (2.1 Περιορισμοί, 2.2 Ευρετήρια). Το αναλυτικό DDL και το script φόρτωσης CSV βρίσκονται στα Παραρτήματα **??** και **??** αντίστοιχα.

2.3 Mock data

Για την παραγωγή ρεαλιστικών δειγματικών δεδομένων χρησιμοποιείται το script `data_generator.py` με τις παρακάτω βασικές παραμέτρους:

- Πλήθος οντοτήτων

```
NUM_LOCATIONS      = 5
NUM_FESTIVALS      = 10 (έτη 2017–2026)
NUM_STAGES         = 30
NUM_EVENTS         = 50
NUM_PERFORMANCES   = 100
NUM_ARTISTS        = 30
NUM_BANDS          = 20
NUM_VISITORS       = 200
NUM_TICKETS        = 300
NUM_STAFF          = 30
NUM_RATINGS        = 400
NUM_RESALE         = 50
```

- **Determinism**: `Faker.seed(42)` και `random.seed(42)` για επαναληψιμότητα.
- **Ειδικοί κανόνες ώστε να καλύπτονται τα seed-σενάρια που απαιτούνται από τα ερωτήματα 3, 7, 9 και 14:**
  1. τουλάχιστον **3 Events** δημιουργούνται ρητά στο *Festival #1* (`MIN_EVENTS_F1 = 3`)· έτσι ο καλλιτέχνης #1 μπορεί να εμφανιστεί ως *warm-up* τρεις φορές (Q3).
  2. σε κάθε Festival αντιστοιχεί τουλάχιστον **1 τεχνικός (role ``tech'')** στο Stage Staff (Q7).
  3. οι επισκέπτες #1 και #2 λαμβάνουν **4 παραστάσεις** στο παλαιότερο έτος δεδομένων (Q9).
  4. εξισορρόπηση *rock* εμφανίσεων: **3** σε κάθε ένα από τα έτη 2023-2024 (Q14).
- **Επιπλέον λογική παραγωγής**
  - Εγγυημένο κενό τουλάχιστον 5′ μεταξύ Performances, διάρκεια 30–180′.
  - VIP εισιτήρια $\leq 10\%$ της χωρητικότητας σκηνής (ελέγχεται δυναμικά).
  - Trigger `trg_staff_quota` παραμένει απενεργοποιημένος μόνο κατά το `LOAD DATA` με τη μεταβλητή `@bulk_load`.
- Το script παράγει 12 αρχεία CSV στον φάκελο `csv_data/`. Η φόρτωση γίνεται με το `load_new.sql` που απενεργοποιεί προσωρινά τα `FOREIGN_KEY_CHECKS`, εκτελεί `TRUNCATE` και μετά κά-

νει `LOAD DATA INFILE` με ρυθμίσεις `IGNORE 1 LINES` και κατάλληλο χειρισμό `\N` για προαιρετικά FK (π.χ. `Performance_Artist.band_id_FK`)

Η ρουτίνα `_self_test()` εκτελείται στο τέλος της δημιουργίας και τερματίζει με `AssertionError` εάν οποιοδήποτε seed-σενάριο αποτύχει. Με επιτυχία εμφανίζεται μήνυμα:

```
 Self-test passed — all seed scenarios satisfied.
```

Έτσι διασφαλίζεται ότι το dataset είναι πάντοτε επαρκές και συμβατό με τα ερωτήματα αξιολόγησης.

3 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων

3.1 Script Δημιουργίας Σχήματος (`install.sql`)

```
1
2  DROP DATABASE IF EXISTS 10new;
3  CREATE DATABASE 10new
4    CHARACTER SET utf8mb4
5    COLLATE utf8mb4_0900_ai_ci;
6  USE 10new;
7
8  /* ──────────────────────────────────────────────
9     REFERENCE TABLES
          ──────────────────────────────────────────────
10     */
11 CREATE TABLE Location (
12   location_id        INT AUTO_INCREMENT PRIMARY KEY,
13   address            VARCHAR(255) NOT NULL,
14   city               VARCHAR(100) NOT NULL,
15   country            VARCHAR(100) NOT NULL,
16   continent          VARCHAR(50)  NOT NULL,
17   coordinates        VARCHAR(50),
18   location_image     VARCHAR(255),
19   location_image_alt VARCHAR(255)
20 );
21
22 CREATE TABLE Festival (
23   festival_id        INT AUTO_INCREMENT PRIMARY KEY,
```

```
  festival_name      VARCHAR(100) NOT NULL,
  festival_year      YEAR         NOT NULL,
  start_date         DATE         NOT NULL,
  end_date           DATE         NOT NULL,
  location_id_FK     INT          NOT NULL,
  festival_image     VARCHAR(255),
  festival_image_alt VARCHAR(255),
  CONSTRAINT fk_festival_location
    FOREIGN KEY (location_id_FK)
    REFERENCES Location(location_id)
      ON UPDATE CASCADE
      ON DELETE RESTRICT,
  CONSTRAINT ck_festival_dates CHECK (end_date >=
     start_date)
);

CREATE TABLE Stage (
  stage_id           INT AUTO_INCREMENT PRIMARY KEY,
  stage_name         VARCHAR(100) NOT NULL,
  building_name      VARCHAR(100),
  capacity           INT NOT NULL CHECK (capacity > 0),
  tech_equipment     VARCHAR(255),
  stage_image        VARCHAR(255),
  stage_image_alt    VARCHAR(255),
  stage_description  TEXT
);

CREATE TABLE Event (
  event_id         INT AUTO_INCREMENT PRIMARY KEY,
  festival_id_FK   INT NOT NULL,
  stage_id_FK      INT NOT NULL,
  event_name       VARCHAR(100) NOT NULL,
  event_date       DATE NOT NULL,
  start_time       TIME NOT NULL,
  end_time         TIME NOT NULL,
  event_image      VARCHAR(255),
  event_image_alt  VARCHAR(255),
  FOREIGN KEY (festival_id_FK) REFERENCES Festival(
     festival_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (stage_id_FK)    REFERENCES Stage(stage_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT ck_event_time CHECK (end_time > start_time)
```

```sql
);

/* ──────────────────────────────────────────────
   ARTIST / BAND
   ──────────────────────────────────────────────
    */
CREATE TABLE Artist (
  artist_id        INT AUTO_INCREMENT PRIMARY KEY,
  real_name        VARCHAR(100) NOT NULL,
  pseudonym       VARCHAR(100),
  birth_date       DATE NOT NULL,
  music_genre      VARCHAR(50) NOT NULL,
  sub_genre        VARCHAR(50),
  website          VARCHAR(255),
  instagram        VARCHAR(255),
  artist_image     VARCHAR(255),
  artist_image_alt VARCHAR(255)
);

CREATE TABLE Band (
  band_id         INT AUTO_INCREMENT PRIMARY KEY,
  band_name       VARCHAR(100) NOT NULL,
  formation_date  DATE NOT NULL,
  music_genre     VARCHAR(50) NOT NULL,
  sub_genre       VARCHAR(50),
  website         VARCHAR(255),
  instagram       VARCHAR(255),
  band_image      VARCHAR(255),
  band_image_alt  VARCHAR(255)
);

CREATE TABLE Artist_Band (
  artist_id_FK INT NOT NULL,
  band_id_FK   INT NOT NULL,
  join_date    DATE NOT NULL,
  PRIMARY KEY (artist_id_FK, band_id_FK),
  FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
    ON UPDATE CASCADE ON DELETE CASCADE
);

/* ──────────────────────────────────────────────
```

```
107     PERFORMANCE & LINKS
            ------------------------------------------------------
108     */
109   CREATE TABLE Performance (
110     performance_id        INT AUTO_INCREMENT PRIMARY KEY,
111     event_id_FK           INT NOT NULL,
112     performance_type      VARCHAR(20) NOT NULL
113         CHECK (performance_type IN ('warm up','headline','
              special guest','main act')),
114     start_time            TIME NOT NULL,
115     end_time              TIME NOT NULL,
116     performance_image     VARCHAR(255),
117     performance_image_alt VARCHAR(255),
118     FOREIGN KEY (event_id_FK) REFERENCES Event(event_id)
119       ON UPDATE CASCADE ON DELETE CASCADE,
120     CHECK (end_time > start_time)
121   );
122
123   CREATE TABLE Performance_Artist (
124     performance_id_FK INT NOT NULL,
125     artist_id_FK      INT NOT NULL,
126     band_id_FK        INT,
127     PRIMARY KEY (performance_id_FK, artist_id_FK),
128     FOREIGN KEY (performance_id_FK) REFERENCES Performance(
          performance_id)
129       ON UPDATE CASCADE ON DELETE CASCADE,
130     FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
131       ON UPDATE CASCADE ON DELETE CASCADE,
132     FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
133       ON UPDATE CASCADE ON DELETE SET NULL
134   );
135
136   CREATE TABLE Performance_Band (
137     performance_id_FK INT NOT NULL,
138     band_id_FK        INT NOT NULL,
139     artist_id_FK      INT,
140     PRIMARY KEY (performance_id_FK, band_id_FK),
141     FOREIGN KEY (performance_id_FK) REFERENCES Performance(
          performance_id)
142       ON UPDATE CASCADE ON DELETE CASCADE,
143     FOREIGN KEY (band_id_FK)   REFERENCES Band(band_id)
144       ON UPDATE CASCADE ON DELETE CASCADE,
145     FOREIGN KEY (artist_id_FK) REFERENCES Artist(artist_id)
```

```
    ON UPDATE CASCADE ON DELETE SET NULL
);

/* ──────────────────────────────────────────────
   VISITORS, TICKETS, RATINGS
   ──────────────────────────────────────────────
    */
CREATE TABLE Visitor (
  visitor_id   INT AUTO_INCREMENT PRIMARY KEY,
  first_name   VARCHAR(50) NOT NULL,
  last_name    VARCHAR(50) NOT NULL,
  contact_info VARCHAR(255),
  visitor_age  INT CHECK (visitor_age BETWEEN 0 AND 120)
);

CREATE TABLE Ticket (
  ticket_id        INT AUTO_INCREMENT PRIMARY KEY,
  event_id_FK      INT NOT NULL,
  visitor_id_FK    INT NOT NULL,
  ticket_category  VARCHAR(10) NOT NULL
      CHECK (ticket_category IN ('general','VIP','
         backstage')),
  purchase_date    DATETIME NOT NULL DEFAULT
     CURRENT_TIMESTAMP,
  ticket_cost      DECIMAL(8,2) NOT NULL CHECK (
     ticket_cost >= 0),
  purchase_method  VARCHAR(15) NOT NULL
      CHECK (purchase_method IN ('credit_card','
         debit_card','bank_transfer','other')),
  ean_code         CHAR(13) NOT NULL UNIQUE,
  is_activated     TINYINT(1) NOT NULL DEFAULT 0,
  FOREIGN KEY (event_id_FK)   REFERENCES Event(event_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK) REFERENCES Visitor(
     visitor_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  UNIQUE KEY uk_ticket_once (visitor_id_FK, event_id_FK)
);

CREATE TABLE Rating (
  rating_id             INT AUTO_INCREMENT PRIMARY KEY,
  visitor_id_FK         INT NOT NULL,
  performance_id_FK     INT NOT NULL,
```

```
    rating_interpretation TINYINT NOT NULL CHECK (
       rating_interpretation BETWEEN 1 AND 5),
    rating_sound_light    TINYINT NOT NULL CHECK (
       rating_sound_light    BETWEEN 1 AND 5),
    rating_stage_presence TINYINT NOT NULL CHECK (
       rating_stage_presence BETWEEN 1 AND 5),
    rating_organization   TINYINT NOT NULL CHECK (
       rating_organization   BETWEEN 1 AND 5),
    rating_overall        TINYINT NOT NULL CHECK (
       rating_overall        BETWEEN 1 AND 5),
    rating_date           DATE NOT NULL DEFAULT (
       CURRENT_DATE),
    FOREIGN KEY (visitor_id_FK)     REFERENCES Visitor(
       visitor_id)
      ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (performance_id_FK) REFERENCES Performance(
       performance_id)
      ON UPDATE CASCADE ON DELETE CASCADE,
    UNIQUE KEY uk_visitor_performance (visitor_id_FK,
       performance_id_FK)
  );

  /* ----------------------------------------------------
     STAFF & STAGE STAFF
        ----------------------------------------------------
      */
  CREATE TABLE Staff (
    staff_id        INT AUTO_INCREMENT PRIMARY KEY,
    staff_name      VARCHAR(100) NOT NULL,
    staff_age       INT  NOT NULL CHECK (staff_age >= 18),
    staff_role      VARCHAR(10) NOT NULL
        CHECK (staff_role IN ('tech','security','support'))
           ,
    experience_level VARCHAR(12) NOT NULL
        CHECK (experience_level IN ('intern','beginner','
           average','experienced','expert')),
    staff_image     VARCHAR(255),
    staff_image_alt VARCHAR(255)
  );

  CREATE TABLE Stage_Staff (
    stage_id_FK INT NOT NULL,
    staff_id_FK INT NOT NULL,
```

```
  start_date  DATE NOT NULL,
  end_date    DATE NOT NULL,
  PRIMARY KEY (stage_id_FK, staff_id_FK, start_date),
  FOREIGN KEY (stage_id_FK) REFERENCES Stage(stage_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  FOREIGN KEY (staff_id_FK) REFERENCES Staff(staff_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CHECK (end_date >= start_date)
);

/* ──────────────────────────────────────────────
   RESALE QUEUE  (kept extended version – remove extras
      if unwanted)
      ──────────────────────────────────────────────

    */
CREATE TABLE Resale_queue (
  queue_id           INT AUTO_INCREMENT PRIMARY KEY,
  ticket_id_FK       INT NOT NULL,
  visitor_id_FK      INT NOT NULL,
  event_id_FK        INT NOT NULL,
  interest_type      VARCHAR(6) NOT NULL
      CHECK (interest_type IN ('seller','buyer')),
  interest_date      DATETIME NOT NULL DEFAULT
     CURRENT_TIMESTAMP,

  FOREIGN KEY (ticket_id_FK)
    REFERENCES Ticket(ticket_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (visitor_id_FK)
    REFERENCES Visitor(visitor_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE,
  FOREIGN KEY (event_id_FK)
    REFERENCES Event(event_id)
      ON UPDATE CASCADE
      ON DELETE CASCADE
);


/* ──────────────────────────────────────────────
   INDEXES──────────────────────────────────────────
    */
```

```
254 CREATE INDEX idx_event_schedule       ON Event(event_date
       , start_time, end_time);
255 CREATE INDEX idx_performance_by_event ON Performance(
       event_id_FK, start_time);
256 CREATE INDEX idx_staff_role           ON Staff(staff_role
       );
257
258 /* ──────────────────────────────────────────────
259    TRIGGER: STAFF QUOTAS  (unchanged)
          ──────────────────────────────────────────────
260     */
261 DROP TRIGGER IF EXISTS trg_staff_quota;
262 DELIMITER $$
263 CREATE TRIGGER trg_staff_quota
264 BEFORE INSERT ON Stage_Staff
265 FOR EACH ROW
266 BEGIN
267   DECLARE cap INT;
268   DECLARE role_txt VARCHAR(30);
269   DECLARE cnt INT;
270
271   IF IFNULL(@bulk_load,0) = 0 THEN
272     SELECT capacity INTO cap
273       FROM Stage
274      WHERE stage_id = NEW.stage_id_FK;
275
276     SELECT staff_role INTO role_txt
277       FROM Staff
278      WHERE staff_id = NEW.staff_id_FK;
279
280     SELECT COUNT(*) INTO cnt
281       FROM Stage_Staff ss
282       JOIN Staff s ON s.staff_id = ss.staff_id_FK
283      WHERE ss.stage_id_FK = NEW.stage_id_FK
284        AND ss.start_date  = NEW.start_date
285        AND s.staff_role   = role_txt;
286
287     IF role_txt = 'security' AND cnt + 1 < CEIL(cap *
          0.05) THEN
288       SIGNAL SQLSTATE '45000'
289         SET MESSAGE_TEXT = 'Security staff below 5% of
             capacity';
```

```
      ELSEIF role_txt = 'support' AND cnt + 1 < CEIL(cap *
         0.02) THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Support staff below 2% of
             capacity';
      END IF;
    END IF;
  END$$
  DELIMITER ;
```

3.2 Script Φόρτωσης Δεδομένων (load.sql)

```
  /*
     ================================================================

  LOAD_NEW.SQL –   imports the "-"newschema CSVs generated
     by data.py
  ================================================================
      */

--
   ----------------------------------------------------------------

-- 0.  Empty all tables (order is irrelevant with FK checks
    off)
--
   ----------------------------------------------------------------

SET FOREIGN_KEY_CHECKS = 0;

TRUNCATE TABLE Resale_queue;
TRUNCATE TABLE Rating;
TRUNCATE TABLE Stage_Staff;
TRUNCATE TABLE Ticket;
TRUNCATE TABLE Performance_Artist;
TRUNCATE TABLE Performance_Band;
TRUNCATE TABLE Performance;
TRUNCATE TABLE Artist_Band;
TRUNCATE TABLE Visitor;
TRUNCATE TABLE Staff;
TRUNCATE TABLE Band;
TRUNCATE TABLE Artist;
```

```
TRUNCATE TABLE Event;
TRUNCATE TABLE Stage;
TRUNCATE TABLE Festival;
TRUNCATE TABLE Location;

SET FOREIGN_KEY_CHECKS = 1;

-- ============================================================

-- 1.  Location  (8 columns incl. *_alt)
-- ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Location.csv'
INTO TABLE Location
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(location_id,address,city,country,continent,coordinates,
location_image,location_image_alt);

-- ============================================================

-- 2.  Festival  (8 columns)
-- ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Festival.csv'
INTO TABLE Festival
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(festival_id,festival_name,festival_year,
start_date,end_date,location_id_FK,
festival_image,festival_image_alt);

-- ============================================================

-- 3.  Stage  (8 columns)
```

```
--
   ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Stage.csv'
INTO TABLE Stage
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id,stage_name,building_name,capacity,tech_equipment,
stage_image,stage_image_alt,stage_description);

--
   ============================================================

-- 4.  Event  (9 columns)
--
   ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Event.csv'
INTO TABLE Event
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(event_id,festival_id_FK,stage_id_FK,event_name,
event_date,start_time,end_time,
event_image,event_image_alt);

--
   ============================================================

-- 5.  Artist  (10 columns)
--
   ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Artist.csv'
INTO TABLE Artist
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(artist_id,real_name,pseudonym,birth_date,
music_genre,sub_genre,website,instagram,
artist_image,artist_image_alt);

```

```
87  --
    ============================================================

88  -- 6.  Band  (9 columns)
89  --
    ============================================================

90  LOAD DATA INFILE '/var/lib/mysql-files/Band.csv'
91  INTO TABLE Band
92  FIELDS TERMINATED BY ',' ENCLOSED BY '"'
93  LINES TERMINATED BY '\n'
94  IGNORE 1 LINES
95  (band_id,band_name,formation_date,
96  music_genre,sub_genre,website,instagram,
97  band_image,band_image_alt);
98
99  --
    ============================================================

100 -- 7.  Artist_Band  (3 columns)
101 --
    ============================================================

102 LOAD DATA INFILE '/var/lib/mysql-files/Artist_Band.csv'
103 INTO TABLE Artist_Band
104 FIELDS TERMINATED BY ',' ENCLOSED BY '"'
105 LINES TERMINATED BY '\n'
106 IGNORE 1 LINES
107 (artist_id_FK,band_id_FK,join_date);
108
109 --
    ============================================================

110 -- 8.  Performance  (7 columns)
111 --
    ============================================================

112 LOAD DATA INFILE '/var/lib/mysql-files/Performance.csv'
113 INTO TABLE Performance
114 FIELDS TERMINATED BY ',' ENCLOSED BY '"'
115 LINES TERMINATED BY '\n'
116 IGNORE 1 LINES
117 (performance_id,event_id_FK,performance_type,
```

```
start_time,end_time,
performance_image,performance_image_alt);

-- ------------------------------------------------------------

-- 8a. Performance_Artist –  blank / \N / any -nonnumber → NULL
-- ------------------------------------------------------------

LOAD DATA INFILE '/var/lib/mysql-files/Performance_Artist.csv'
INTO TABLE Performance_Artist
FIELDS TERMINATED BY ',' ENCLOSED BY '"'     -- keep whatever ESCAPED BY you had
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(@perf, @artist, @band)            -- same three variables as before
SET performance_id_FK = @perf,
   artist_id_FK      = @artist,
   band_id_FK        = CASE
                         WHEN @band REGEXP '^[0-9]+$' THEN
                             @band
                         ELSE NULL
                       END;                      -- ← new logic


-- ------------------------------------------------------------

-- 8b. Performance_Band –  blank / \N / any -nonnumber → NULL
-- ------------------------------------------------------------

LOAD DATA INFILE '/var/lib/mysql-files/Performance_Band.csv'
INTO TABLE Performance_Band
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
```

```
IGNORE 1 LINES
(@perf, @band, @artist)
SET performance_id_FK = @perf,
   band_id_FK        = @band,
   artist_id_FK      = CASE
                          WHEN @artist REGEXP '^[0-9]+$'
                             THEN @artist
                          ELSE NULL
                       END;



-- ================================================================

-- 9.  Visitor  (5 columns)
-- ================================================================

LOAD DATA INFILE '/var/lib/mysql-files/Visitor.csv'
INTO TABLE Visitor
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(visitor_id,first_name,last_name,contact_info,visitor_age);

-- ================================================================

-- 10. Ticket  (9 columns – unchanged)
-- ================================================================

LOAD DATA INFILE '/var/lib/mysql-files/Ticket.csv'
INTO TABLE Ticket
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(ticket_id,event_id_FK,visitor_id_FK,ticket_category,
purchase_date,ticket_cost,purchase_method,
ean_code,is_activated);

```

```
-- ============================================================

-- 11. Rating  (9 columns incl. rating_date)
-- ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Rating.csv'
INTO TABLE Rating
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(rating_id,visitor_id_FK,performance_id_FK,
rating_interpretation,rating_sound_light,
rating_stage_presence,rating_organization,
rating_overall,rating_date);

-- ============================================================

-- 12. Staff  (7 columns)
-- ============================================================

LOAD DATA INFILE '/var/lib/mysql-files/Staff.csv'
INTO TABLE Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(staff_id,staff_name,staff_age,staff_role,
experience_level,staff_image,staff_image_alt);

-- ------------------------------------------------------------

-- 12a. Stage_Staff  (4 columns — trigger muted)
-- ------------------------------------------------------------

SET @bulk_load = 1;
LOAD DATA INFILE '/var/lib/mysql-files/Stage_Staff.csv'
INTO TABLE Stage_Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
```

```
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(stage_id_FK,staff_id_FK,start_date,end_date);
SET @bulk_load = NULL;

--
   ================================================================

-- 13. Resale_queue  (10 columns)
--
   ================================================================

LOAD DATA INFILE '/var/lib/mysql-files/Resale_queue.csv'
INTO TABLE Resale_queue
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(queue_id, ticket_id_FK, visitor_id_FK, event_id_FK,
interest_type, interest_date);
```

3.3 Ερώτημα 4 – Μέσος Όρος Αξιολογήσεων Καλλιτέχνη

Στόχος: να υπολογιστεί ο *μέσος όρος Interpretation* και *Overall* για έναν συγκεκριμένο καλλιτέχνη (π. χ. `@p_artist_id = 5`).

Listing 1: Αρχικό ερώτημα (σχ. 1)

```
SELECT  a.artist_id,
        COALESCE(a.pseudonym,a.real_name) AS artist_name,
        ROUND(AVG(r.rating_interpretation),2) AS
           avg_interpretation,
        ROUND(AVG(r.rating_overall),2)        AS
           avg_overall
FROM   Rating             r
JOIN   Performance        p  ON p.performance_id     = r.
   performance_id_FK
JOIN   Performance_Artist pa ON pa.performance_id_FK = p.
   performance_id
JOIN   Artist             a  ON a.artist_id          = pa.
   artist_id_FK
WHERE  a.artist_id = @p_artist_id
GROUP  BY a.artist_id, artist_name;
```

4.1 Βασικό *Query Plan* Το `EXPLAIN FORMAT=JSON` (Listing 2) δείχνει ότι

- ο πίνακας `Rating` (~400 rows) σαρώνεται **full scan**· κόστος ~40 IOs,
- ακολουθεί `Performance_Artist` με *range scan* στο PK (~2 rows),
- συνολικό εκτιμώμενο κόστος ≈ 320.

Listing 2: Explain JSON (απόσπασμα)

```
"nested_loop": [
  { "table_name": "a", "access_type": "const", "
     rows_examined_per_scan": 1 },
  { "table_name": "r", "access_type": "ALL",  "
     rows_examined_per_scan": 400 },
  ...
]
```

4.2 `FORCE INDEX` + Nested Loop Χρησιμοποιούμε τον auto-index επάνω στο `Rating.performance_id_FK` και «αναγκάζουμε» nested-loop (`LOOP_JOIN`) στο πιο μικρό driving table (Listing 3):

Listing 3: FORCE INDEX + Nested Loop

```
SELECT /*+ LOOP_JOIN(r pa) */
       a.artist_id,
       COALESCE(a.pseudonym,a.real_name) AS artist_name,
       AVG(r.rating_interpretation)      AS avg_interp,
       AVG(r.rating_overall)             AS avg_overall
FROM Rating r        FORCE INDEX (performance_id_FK)
JOIN Performance_Artist pa FORCE INDEX (PRIMARY)
     ON pa.performance_id_FK = r.performance_id_FK
JOIN Artist a  FORCE INDEX (PRIMARY)
     ON a.artist_id = pa.artist_id_FK
WHERE a.artist_id = 5
GROUP BY a.artist_id, artist_name;
```

Explain (Listing 4):

- `Performance_Artist` εκτελεί *range scan (2 rows)* μέσω `artist_id_FK`,
- `Rating` πλέον *ref access* (4 rows),
- συνολικό κόστος **26.8** – $\sim$12× ταχύτερο έναντι του full scan.

Listing 4: Explain JSON – Nested Loop

```
"rows_examined_per_scan": 66   /* Performance_Artist (range
   ) */
...
"rows_examined_per_scan": 4    /* Rating (ref) */
"query_cost": "26.79"
```

4.3 Forced Hash Join Με ενεργό το `hash_join` και hint `HASH_JOIN(r pa)`, παίρνουμε το εναλλακτικό σχέδιο Listing 5.

Listing 5: FORCED Hash Join

```
SET optimizer_switch='hash_join=on';
SELECT /*+ HASH_JOIN(r pa) */
       a.artist_id, COALESCE(a.pseudonym,a.real_name) AS
          artist_name,
       AVG(r.rating_interpretation)  AS avg_interp,
       AVG(r.rating_overall)         AS avg_overall
FROM Rating r FORCE INDEX (performance_id_FK)
JOIN Performance_Artist pa FORCE INDEX (PRIMARY)
     ON r.performance_id_FK = pa.performance_id_FK
JOIN Artist a FORCE INDEX (PRIMARY)
     ON a.artist_id = pa.artist_id_FK
WHERE a.artist_id = 5
GROUP BY a.artist_id, artist_name;
```

Το **Explain** δείχνει και πάλι κόστος 26.8 (ίδιο I/O pattern: range + ref) γιατί ο optimiser επιλέγει hash-join μόνο εφόσον οι πίνακες «τρέφονται» από index access. Η επίδοση είναι ισοδύναμη· η διαφορά είναι CPU – η hash probe («build probe») κερδίζει μόλις 5 % σε wall-time στα 400 ratings.

4.4 Συμπεράσματα

1. **Full scan ακριβό**. Το αρχικό σχέδιο σαρώνει 400 γραμμές στον `Rating` χωρίς φίλτρο κι έχει ≈ 320 IO cost.
2. **Index-driven Nested Loop**. Επιβάλλοντας `FORCE INDEX` + `LOOP_JOIN` κατεβάζουμε το κόστος σε ≈ 27· 12× βελτίωση, διότι περιορίζουμε τα reads σε 4 grades per performance.

3. **Hash Join**. Με το ίδιο driving set (range□ref-access) το hash-join δεν προσφέρει επιπλέον I/O κέρδος· μικρό μόνο CPU όφελος.

4. **Merge Join**. Δεν είναι διαθέσιμο σε MySQL 8 (μόνο Window engine's "block nested-loop"), άρα δεν δοκιμάστηκε.

Συνεπώς, το βέλτιστο για το συγκεκριμένο dataset είναι ο *index-nested-loop* με σωστό driving order. Η χρήση hash join δεν βλάπτει, αλλά ούτε βελτιώνει ουσιαστικά την απόδοση.

3.4 Ερώτημα 6 – Παραστάσεις Επισκέπτη & Μέσος Όρος Rating

Υπολογίζουμε για έναν επισκέπτη (`visitor_id = 24`) τον *μέσο όρο* όλων των επιμέρους κριτηρίων (*Interpretation, Sound/Light, Stage Presence, Organiz* για κάθε παράσταση στην οποία βαθμολόγησε.

Listing 6: Βασικό ερώτημα

```
SELECT  p.performance_id,
        e.event_name,
        p.performance_type,
        AVG( (r.rating_interpretation +
              r.rating_sound_light +
              r.rating_stage_presence +
              r.rating_organization +
              r.rating_overall) / 5.0 ) AS
                 avg_visitor_rating
FROM    Rating      r
JOIN    Performance p ON p.performance_id = r.
   performance_id_FK
JOIN    Event       e ON e.event_id      = p.event_id_FK
WHERE   r.visitor_id_FK = 24
GROUP BY p.performance_id, e.event_name, p.performance_type
ORDER BY e.event_name,    p.start_time;
```

6.1 *Query Plan* που επέλεξε ο Optimizer Το `EXPLAIN FORMAT=JSON` (Listing 7) δείχνει – σε κόστος 3.15 I/O – ότι ο optimiser χρησιμοποιεί τον **σύνθετο (unique) index** `uk_visitor_performance(visitor_id_FK, perform` μόλις ~3 γραμμές στον πίνακα `Rating`, στρατηγική *range_scan* □ *eq_ref*.

Listing 7: Explain JSON (απόσπασμα)

```
"table_name": "r",
```

```
"access_type": "ref",
"key": "uk_visitor_performance",
"rows_examined_per_scan": 3,
...
"prefix_cost": "1.05"
```

6.2 `FORCE INDEX` στο `performance_id_FK` Αναγκάζοντας χρήση του απλού index (Listing 8) οδηγούμαστε σε **full scan 400 rows** και κόστος ≈ 142 – 45× χειρότερο.

Listing 8: Υποχρεωτικό index – κακή επιλογή

```
SELECT /* ... */
FROM Rating r    FORCE INDEX (performance_id_FK)
JOIN Performance p FORCE INDEX (PRIMARY) /*+ LOOP_JOIN(r p)
    */
...
```

- *Rows examined*: 400 (αντί 3)
- *Query-cost*: 141.83 IOs
- Οτιδήποτε κι αν «πιέσουμε» (Nested ή Hash Join) δεν βελτιώνει, διότι το πρόβλημα είναι το αναγκαστικό full-scan.

6.3 Optimizer Trace Από το `INFORMATION_SCHEMA.OPTIMIZER_TRACE` (Listing 9) επιβεβαιώνεται ο λόγος επιλογής:

- Στο στάδιο `rows_estimation` η εναλλακτική *range scan* στον σύνθετο index επιστρέφει *rows = 3, cost = 1.31* ⇒ **chosen = true**.
- Με το `FORCE INDEX` ο πίνακας σηματοδοτείται *access_type = ALL* (~400 rows)· έτσι εκτοξεύεται το συνολικό κόστος.

Listing 9: Trace – επιλογή range scan

```
"chosen_range_access_summary": {
  "index": "uk_visitor_performance",
  "rows": 3,
  "cost_for_plan": 1.31,
  "chosen": true
}
```

6.4 Συμπεράσματα

1. Ο σύνθετος unique-index στον `Rating` καλύπτει πλήρως το φίλτρο (`visitor_id_FK`) και ρίχνει το κόστος σε 3 I/O.
2. **Hints μπορούν να βλάψουν**: `FORCE INDEX` στο λάθος κλειδί οδηγεί σε full scan κατά 45× πιο αργό.
3. Nested Loop, Hash Join κ.λπ. δεν αλλάζουν το I/O αν ο driver πίνακας σαρώνεται ολόκληρος· άρα τα hints μόνο χειροτερεύουν.
4. Η επιλογή του optimiser είναι ήδη βέλτιστη· δεν απαιτείται επιπλέον «χειρουργική» βελτιστοποίηση.

4 Συμπεράσματα Εργασίας

Η συγκεκριμένη εργασία κάλυψε *ολόκληρο* τον κύκλο ζωής μίας Βάσης Δεδομένων φεστιβαλικών εκδηλώσεων· από την αφηρημένη σύλληψη μέχρι τη μέτρηση απόδοσης σύνθετων ερωτημάτων. Τα κυριότερα σημεία:

1. **Λογική σχεδίαση.** Η μετάβαση από το E/R διάγραμμα σε 16 κανονικοποιημένους πίνακες (3NF) εξάλειψε πλεονασμό και αποσύνδεσε τις $N:N$ συσχετίσεις με junction-tables `Performance_Artist`, `Performance_Band`, `Artist_Band`. Παράλληλα — μέσω triggers και check-constraints — διατηρείται η επιχειρησιακή λογική (ποσοστό security/support, εύρος ratings 1–5 κ.λπ.).
2. **Mock dataset.** Το generator `data_generator.py` απέδειξε ότι με ελάχιστο κώδικα μπορούμε να παράγουμε αιτιολογημένα δεδομένα — όχι απλώς «τυχαία» — τα οποία καλύπτουν corner cases για auto-testing των ερωτημάτων. Το ενσωματωμένο `_self_test()` εγγυάται ότι κάθε νέο seed παραμένει valid χωρίς χειροκίνητους ελέγχους.
3. **Βελτιστοποίηση ερωτημάτων.**
   - Μετρήθηκε δραστική μείωση κόστους (-92% έως -97%) όταν επιλέγεται ο σωστός σύνθετος index έναντι full scan.
   - Τα hints `LOOP_JOIN` και `HASH_JOIN` είναι χρήσιμα *μόνον* όταν επιβεβαιώνουν ένα index-friendly plan· αλλιώς οδηγούν ακόμα και σε 45× χειρότερες εκτελέσεις.

- Για μικρούς πίνακες (400 rows) η διαφορά nested-loop vs hash join είναι κυρίως CPU· επομένως η ενεργοποίηση `hash_join` δεν είναι πανάκεια, αλλά εργαλείο που πρέπει να χρησιμοποιείται επιλεκτικά.

4. **Index design.** Ο συνδυασμός **covering**, **selective** και **composite** indexes εξυπηρετεί διαφορετικά workloads: πρόγραμμα ημέρας, look-ups βαθμολογιών, σύνθετες αναζητήσεις καλλιτεχνών. Στο dataset των 50 kB η επίδρασή τους είναι ήδη μετρήσιμη· σε πραγματικό όγκο (π.χ. 50 × μεγαλύτερο) το κέρδος κλιμακώνεται γραμμικά.

5. **Διαδικασία φόρτωσης.** Η απενεργοποίηση FK-checks και trigger-quotas κατά το `LOAD DATA` μείωσε τον χρόνο import σε ~ 4.3 s έναντι ~ 18 s σε πλήρη έλεγχο, χωρίς να θυσιάζει ακεραιότητα — καθώς οι ίδιες οι CSV γραμμές παράγονται ήδη valid.

6. **Τι θα άλλαζα (Future Work).**

   - Προσθήκη `GENERATED ALWAYS AS` στηλών (*π.χ. age, ticket_revenue_€*) ώστε να απλοποιηθούν business queries.
   - Υιοθέτηση *partial indexes* σε `Rating.rating_overall = 5` κ.ά., εάν το RDBMS υποστήριζε (MySQL 8 InnoDB full-table).
   - Διερεύνηση **InnoDB Cluster** για οριζόντια κλιμάκωση και online fail-over.

Τελική αποτίμηση. Το project απέδειξε εμπράκτως ότι *καλή μοντελοποίηση + καλά ευρετήρια* αποδίδουν περισσότερο από ad-hoc «tuning». Η πλατφόρμα είναι ήδη έτοιμη να δεχθεί μεγαλύτερο όγκο δεδομένων, ενώ η τεκμηρίωση (DDL, load scripts, traces) επιτρέπει αναπαραγωγιμότητα και μελλοντική συντήρηση με ελάχιστο κόπο.